AUK Campus Parking Management System

Marigona Kastrati, Enis Popovic, Anese Kastrati, and Fjolla Beha

Computing and Information Technologies, RIT Kosovo

SWEN 383: Software Design Principles and Patterns

Valon Raca

October 26, 2026

Table of Contents

1. Project Overview 3
2. Intended Users 3
3. Project Scope 3
    3.1 In Scope 3-4
    3.2 Out of Scope 4
    3.3 Assumptions 5
4. Key Features and Business Rules 5-6
5. UML Class Diagram 6-7
6. Sequence Diagram – Reserve a Parking Space 7-8
7. Sequence Diagram – Cancel a Reservation 8-9
8. Sequence Diagram – Admin Manages Parking Availability 9-10
9. Anticipated Design Risks and Planned Design Patterns 10-11
    9.1 Risk 1: Double Booking of Parking Spaces 10-11
    9.2 Risk 2: Incorrect Reservation and Deposit Statuses 11-12
    9.3 Design Summary 12
10. GitHub Repository and Team Contribution 12

1. Project Overview

The AUK Campus Parking Management System is a proposed application designed to make parking at AUK easier to find, reserve, and manage. Students and staff can view recorded parking availability and reserve a space for a specific date and time. Security personnel confirm vehicle arrivals and departures, while administrators manage parking spaces, user accounts, reservations, and operating hours.

The system's main purpose is to reduce booking conflicts, save time, improve parking information accuracy, and make campus parking more organized and convenient.

2. Intended Users

The system supports four main user roles:

User Role	Main Responsibilities
Student	View available parking spaces, reserve a space, view personal reservations, and cancel eligible reservations.
Staff Member	View available spaces, make reservations, view personal reservations, and cancel eligible reservations.
Security Personnel	Verify reservations, confirm vehicle arrivals and departures, and update parking occupancy records.
Administrator	Manage user accounts, parking spaces, reservations, operating hours, and temporarily unavailable spaces.

Table 1. Intended Users and Their Responsibilities

3. Project Scope

3.1 In Scope

The initial version of the system will include:

Feature	Description
User Access	Provide account access and role-based permissions.
Parking Dashboard	Display identified parking spaces and their recorded availability.
Parking Status	Track spaces as Available, Reserved, Occupied, or Out of Service.
Reservations	Allow students and staff to reserve spaces for specific dates and times.

Booking Validation	Check booking rules and prevent overlapping reservations.
Reservation Management	Allow users to view and cancel eligible personal reservations.
Check-In and Check-Out	Allow security personnel to confirm vehicle arrivals and departures.
Simulated Deposit	Record a virtual refundable €1 deposit for each confirmed reservation.
Cancellation and No-Shows	Record virtual refunds or deposit retention according to the rules.
Administration	Allow administrators to manage users, parking spaces, and reservations.
Operating Hours	Allow administrators to configure permitted parking hours.
Space Maintenance	Allow administrators to temporarily mark spaces as unavailable.

Table 2. In-Scope Features and Descriptions

3.2 Out of Scope

The initial version will not include:

Excluded Feature	Reason for Exclusion
Real Payment Processing	The deposit is simulated; no real money is transferred.
Physical Parking Sensors	The system relies on recorded information and security confirmations.
License Plate Recognition	The first version does not require camera-based vehicle identification.
Automated Parking Gates	Physical access-control hardware is excluded.
GPS Tracking and Navigation	Automatic navigation to parking spaces is not part of the initial system.
University System Integration	Integration with existing AUK student or authentication systems is excluded.
AI-Based Predictions	Advanced parking forecasting and AI-based space allocation are not required.

Native Mobile Applications	Separate Android and iOS applications are excluded from the initial version.

Table 3. Out-of-Scope Features and Reasons for Exclusion

3.3 Assumptions

The system will operate based on a set of assumptions about parking spaces, availability records, and the application platform. These assumptions establish the conditions for the system's initial version and are outlined in Table 4.

Area	Assumption
Parking Data	The system uses a predefined collection of identified parking spaces.
Availability	Recorded availability depends on system data and security personnel confirmations.
Application Platform	A responsive web application may be used for the initial prototype.
Deposits	Deposits, refunds, and retention are simulated; no actual payments occur.

Table 4. System Assumptions

4. Key Features and Business Rules

The system will follow these main rules:

No.	Business Rule	Description
1	Reservation Duration	A reservation must have a valid start time before its end time and cannot exceed six hours.
2	Advance Booking	Users can reserve a space up to 24 hours before the scheduled start time.
3	Reservation Limit	Each user can have only one open reservation at a time.
4	Double-Booking Prevention	The same parking space cannot have overlapping reservations.
5	Operating Hours	Reservations must fall within the hours configured by an administrator.

6	Virtual Deposit	Each confirmed reservation records a simulated refundable €1 deposit.
7	Early Cancellation	Cancellation before the scheduled start time results in a virtual deposit refund.
8	Late Cancellation	Cancellation after the scheduled start time but before check-in results in deposit retention.
9	No-Show Handling	If arrival is not confirmed within the ten-minute grace period, the reservation becomes a no-show and the simulated deposit is retained.
10	Occupancy Management	Security personnel confirm arrivals and departures. An occupied space does not automatically become available just because its scheduled end time has passed.

Table 5. Key Features and Business Rules

5. UML Class Diagram

The UML class diagram presents the main classes, attributes, methods, and relationships of the AUK Campus Parking Management System.

The main classes include:

- User**:** Stores common information about system users.
- Student**:** Represents a student using the system.
- StaffMember**:** Represents a staff member using the system.
- SecurityPersonnel**:** Verifies vehicle arrivals and departures.
- Administrator**:** Manages users, parking spaces, reservations, and operating hours.
- Vehicle**:** Stores registered vehicle information.
- ParkingSpace**:** Represents a parking space and manages its availability.
- Reservation**:** Stores reservation details, times, and status.
- DepositRecord**:** Tracks simulated deposit amounts and outcomes.
- OperatingHours**:** Defines the days and times during which reservations are permitted.

The diagram uses inheritance to share common information between the User class and its specialized user classes. Associations connect users, vehicles, reservations, parking spaces, and deposit records.

Enumerations such as UserType, SpaceStatus, ReservationStatus, and DepositStatus define the available user categories and system statuses.

The class diagram provides a structural overview of how the system will manage parking reservations, availability, check-in/check-out, and virtual deposits.

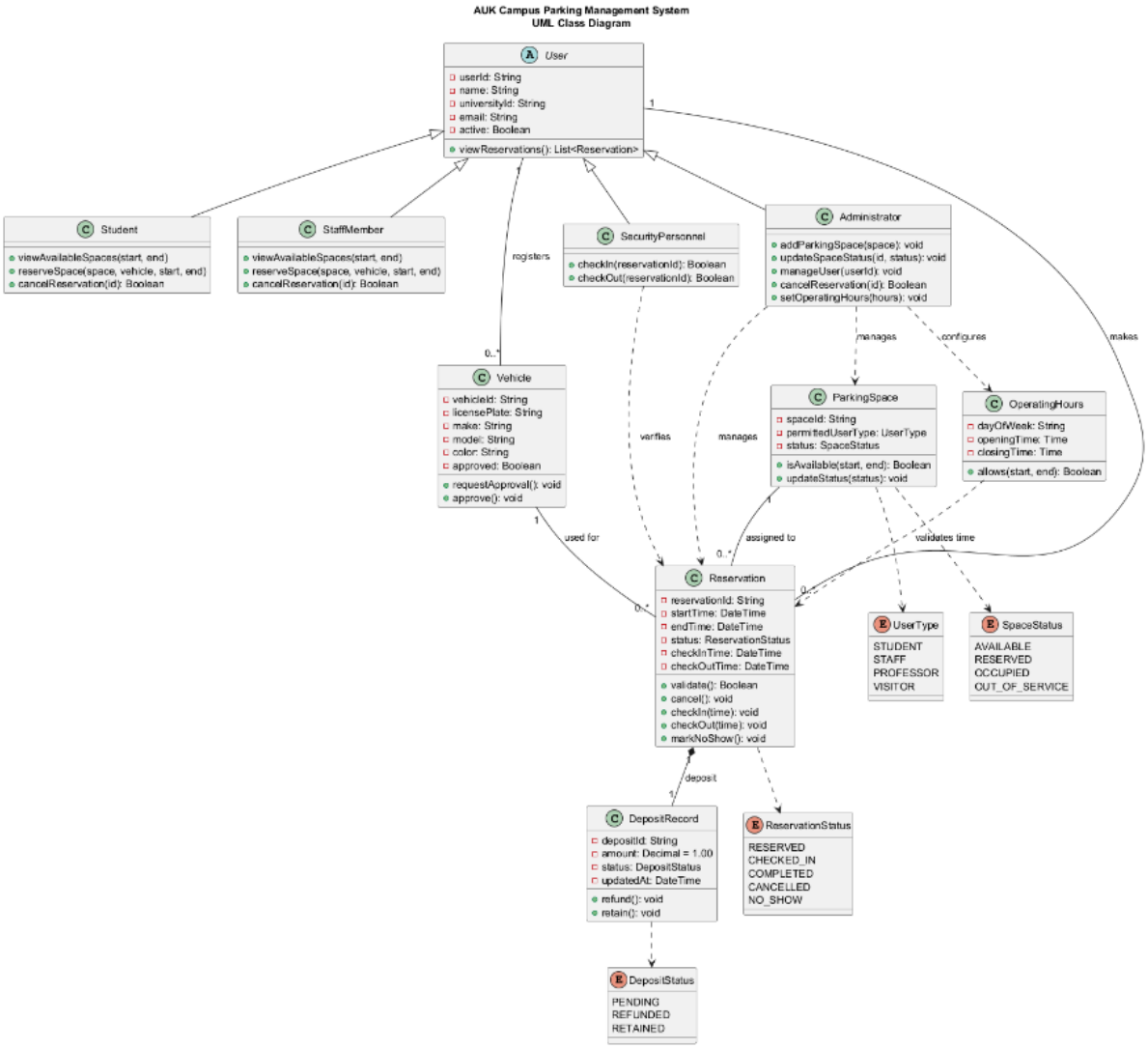


Figure 1. UML Class Diagram of the AUK Campus Parking Management System

6. Sequence Diagram – Reserve a Parking Space

This sequence diagram illustrates how a student requests and reserves a parking space.

The system checks the booking rules, including reservation duration, booking limits, existing reservations, and operating hours. It then checks whether the selected parking space is available.

If the booking rules are invalid, the system displays a validation error. If no suitable space is available, the system informs the user. If the space is available, the system validates the reservation, creates the reservation record, and records the simulated €1 deposit.

Finally, the system confirms the reservation to the student.

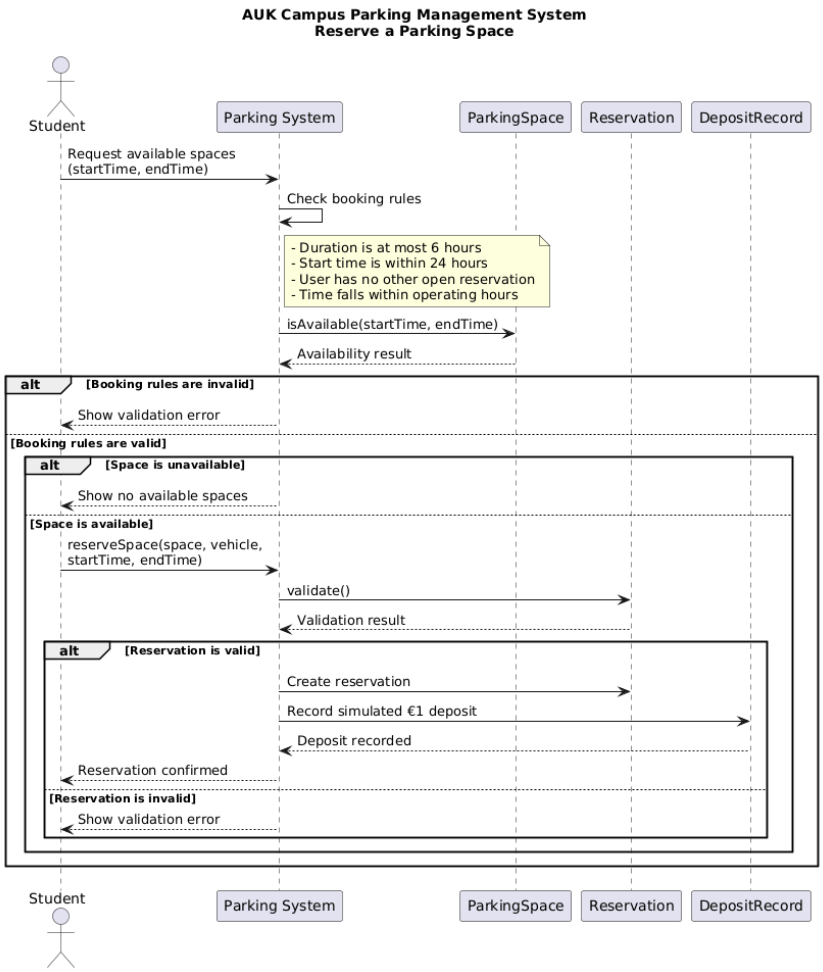


Figure 2. Sequence Diagram for Reserving a Parking Space

7. Sequence Diagram – Cancel a Reservation

This sequence diagram illustrates how a student cancels an existing reservation.

The system locates the reservation and checks whether cancellation is permitted. If the reservation does not exist, the system informs the user. If the reservation has already been cancelled, completed, or checked in, the cancellation request is rejected.

For an eligible reservation, the system compares the current time with the scheduled start time.

- If cancellation occurs before the scheduled start time, the system records a virtual refund of the €1 deposit.
- If cancellation occurs after the scheduled start time but before check-in, the system retains the simulated deposit.

The system then confirms the cancellation and records the appropriate deposit outcome.

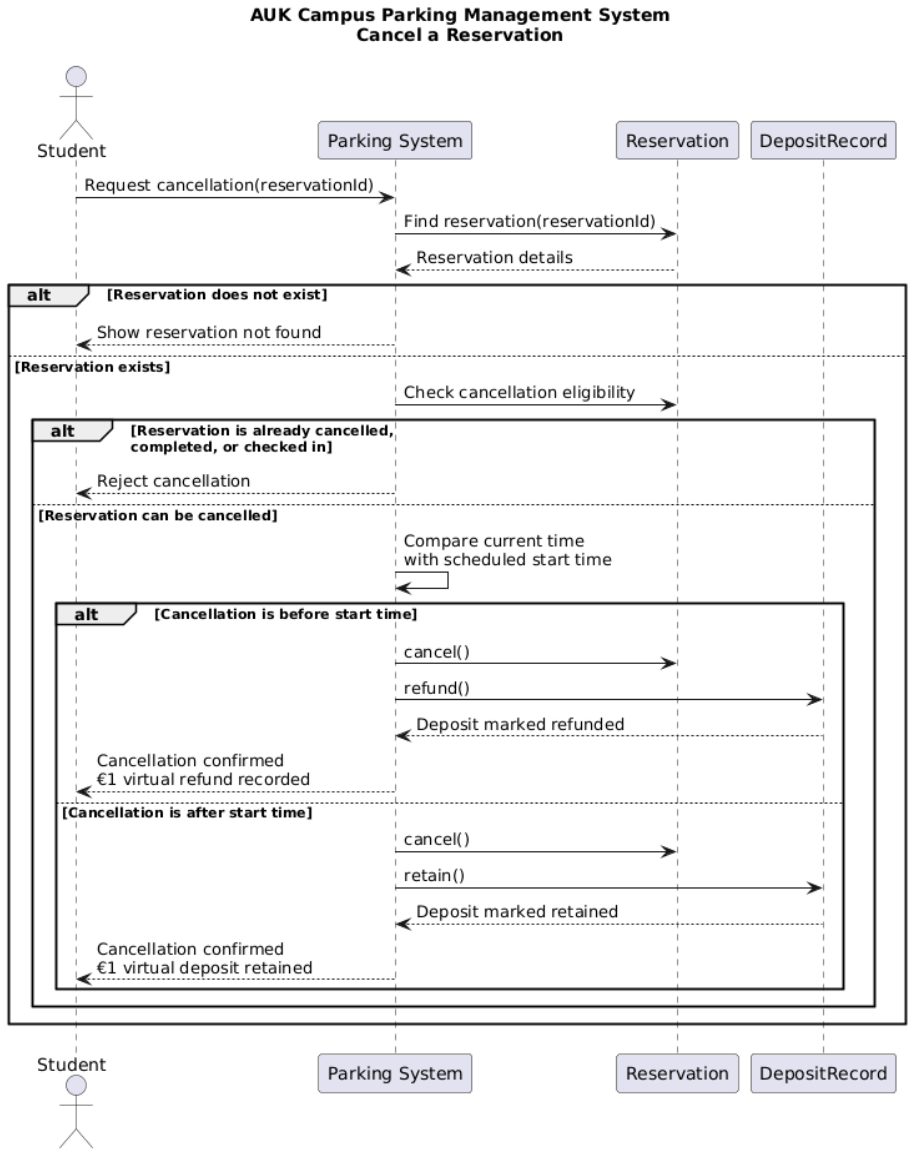


Figure 3. Sequence Diagram for Cancelling a Reservation

8. Sequence Diagram – Admin Manages Parking Availability

This sequence diagram illustrates how an administrator updates the availability status of a parking space.

The system first verifies the administrator's permissions and locates the requested parking space. If the administrator is unauthorized or the space does not exist, the system rejects the request or displays an appropriate message.

If the space exists, the system checks its current status and validates the requested new status. It also checks whether existing reservations may be affected by the change.

If the update is valid and there are no unresolved reservation conflicts, the system updates the parking space status and confirms the change. If the update is invalid or reservations are affected, the system displays an error or requests a review.

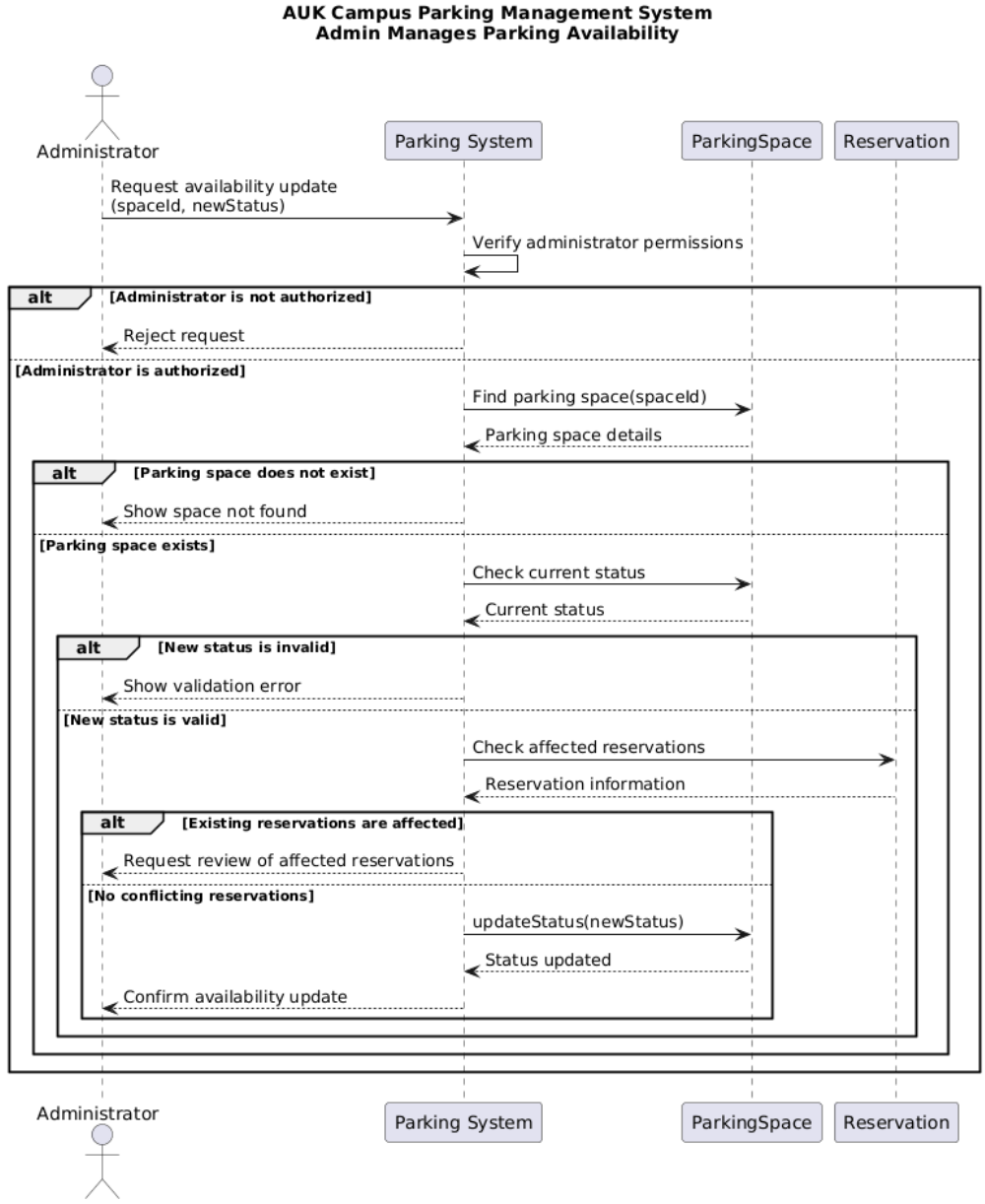


Figure 4. Sequence Diagram for Managing Parking Availability

9. Anticipated Design Risks and Planned Design Patterns

9.1 Risk 1: Double Booking of Parking Spaces

Problem:

Two students may attempt to reserve the same parking space for overlapping periods. If the system does not handle these requests correctly, both reservations could be accepted, resulting in a booking conflict.

Planned Design Pattern: Repository Pattern

The Repository Pattern will separate reservation data operations from the reservation business logic. For example, a ReservationRepository will handle saving and retrieving reservation records rather than mixing database operations directly into the booking logic.

However, the Repository Pattern alone does not prevent double booking. The system must also use appropriate database transactions and conflict checks to ensure that overlapping reservations cannot both be confirmed.

Connection to SOLID and GRASP:

- Single Responsibility Principle (SRP): Reservation rules and database operations have separate responsibilities.
- GRASP Low Coupling**:** Reservation logic does not need to depend directly on the details of how data is stored.

This risk relates primarily to the Reservation and ParkingSpace classes.

9.2 Risk 2: Incorrect Reservation and Deposit Statuses

Problem:

The system must manage reservation states such as Reserved, Checked In, Completed, Cancelled, and No-Show. Incorrect state changes could allow invalid actions, such as cancelling a reservation after check-in, or cause a deposit to be refunded or retained more than once.

Planned Design Pattern: State Pattern

The State Pattern will organize reservation behavior according to the reservation's current state. For example, a reservation that is Reserved may be eligible for cancellation, while a reservation that is Checked In should not allow normal cancellation.

This approach will help organize status-transition rules and prevent invalid operations. Deposit updates must also be recorded consistently.

Connection to SOLID and GRASP:

- Single Responsibility Principle (SRP): State-specific behavior can be separated rather than placing all status conditions in one large method.
- GRASP Polymorphism**:** Different reservation states can respond differently to the same action.

This risk relates to the Reservation, ReservationStatus, DepositRecord, and DepositStatus elements.

9.3 Design Summary

The Repository Pattern is proposed to organize reservation data operations, together with database transactions and conflict checks to prevent double booking. The State Pattern is proposed to organize reservation status transitions and help ensure that only valid actions are allowed.

These patterns are proposed design choices for the initial design baseline. Their implementation will be refined during later project development.

10. GitHub Repository and Team Contribution

GitHub Repository: **CLICK HERE!!!**